

Blog Helper - Projektna dokumentacija

Student: Luka Batarelo

JMBAG: 0067526658

Kolegij: Programsko inženjerstvo

Fakultet informatike u Puli (<https://fipu.unipu.hr/>)

Kolegij: Programsko inženjerstvo (<https://ntankovic.unipu.hr/pi>)

Raspodijeljeni sustavi (<https://ntankovic.unipu.hr/rs>)

Mentor: doc. dr. sc. Nikola Tanković (<https://ntankovic.unipu.hr>)

Sadržaj

Blog Helper - Projektna dokumentacija	1
Sadržaj	2
1. Sažetak	3
2. Uvod	3
3. Motivacija	3
3.1. SWOT analiza	4
3.2. Buduća poboljšanja	4
4. Razrada funkcionalnosti	4
4.1. Use-case dijagram	4
4.2. Dijagrami slijeda	5
4.3. Relacijski model	7
4.4. Klasni dijagram	9
4.5. Prototip sučelja	10
5. Implementacija	10
5.1. Arhitektura sustava	10
5.2. Povezanost komponenti sučelja (frontend)	12
5.3. Backend servisi	12
auth-service (:8001)	12
posts-api (:8002)	13
ai-api (:8003)	13
6. Korisničke upute	14
Pokretanje aplikacije	14
Prijava u aplikaciju	15
Pregled postova (Dashboard)	15
Kreiranje novog posta	16
Prikaz i brisanje posta	17
Uređivanje posta	17

1. Sažetak

Blog Helper je web aplikacija za AI-potpomognuto pisanje blog postova. Aplikacija korisnicima omogućava autentikaciju putem Auth0 OAuth2 protokola te upravljanje blog postovima kroz intuitivno korisničko sučelje. Ključna prednost aplikacije je integracija s lokalnim jezičnim modelom (Ollama / gemma3) koji korisniku pruža mogućnost automatskog generiranja sadržaja na temelju zadanog naslova, poboljšanja postojećeg teksta te predlaganja alternativnih naslova. Aplikacija je izgrađena kao skup mikroservisa: servis za autentikaciju (auth-service), servis za upravljanje objavama (posts-api), servis za AI funkcionalnosti (ai-api) te frontend aplikacija (React).

2. Uvod

Blog Helper je web aplikacija namijenjena svim korisnicima koji se bave pisanjem blogova i žele ubrzati te olakšati taj proces uz pomoć umjetne inteligencije. Ciljna publika su programeri, blogeri i studenti te svi koji redovito produciraju pisani sadržaj te traže alat koji će im pomoći u kreiranju istog, poboljšanju kvalitete tekstova i smanjenju vremena potrebnog za kreiranje novih objava.

Aplikacija se temelji na ideji da pisanje blog objave ne mora biti naporan proces. Korisnik upiše naslov koji ga zanima, a aplikacija automatski generira inicijalni sadržaj koristeći lokalni jezični model. Korisnik zatim može nastaviti uređivati tekst, zatražiti poboljšanja stila ili dobiti prijedloge alternativnih naslova - sve unutar jednog sučelja.

U usporedbi s dostupnim rješenjima, Blog Helper se ističe po nekoliko ključnih prednosti. Koristi lokalni AI model (Ollama s gemma3 modelom) što znači da sadržaj ne napušta korisnikovo računalo i nema troškova API poziva prema vanjskim pružateljima AI usluga. Aplikacija je jednostavna za postavljanje i korištenje, a cijela arhitektura bazirana je na modernim otvorenim tehnologijama (FastAPI, React, SQLite).

3. Motivacija

Tržište alata za pisanje sadržaja uz AI asistenciju u posljednjih je nekoliko godina doživjelo snažan rast. Alati poput Jasper AI, Copy.ai i Notion AI postali su popularni među profesionalnim piscima sadržaja. Međutim, velika većina tih alata oslanja se na komercijalne API-jeve (OpenAI, Anthropic, Google) što podrazumijeva redovite financijske troškove po generiranom tokenu, slanje privatnog sadržaja na vanjske servere te ovisnost o dostupnosti tih usluga. Blog Helper nastoji riješiti taj problem korištenjem lokalnog jezičnog modela. Sve AI operacije odvijaju se lokalno na korisnikovom stroju putem Ollama servera. Ovo je posebno zanimljivo za ljude koji rade s osjetljivim temama te ne žele da njihov sadržaj bude obrađivan od strane vodećih AI kompanija. Ciljano tržište su pojedinačni korisnici - blogeri i programeri koji traže besplatno ili jeftino rješenje za AI-potpomognuto pisanje. Preduvjeti za korištenje aplikacije su instaliran Ollama runtime s

povučenim gemma3 modelom, Auth0 korisnički račun za autentikaciju te stabilna lokalna mrežna veza između servisa.

Razvoj aplikacije odvijao se u kratkom vremenskom roku. Prvo su implementirani backend mikroservisi (auth-service i posts-api), zatim AI servis s podrškom za streaming odgovora, a naposljetku React frontend koji sve to objedinjuje u funkcionalno korisničko sučelje.

3.1. SWOT analiza

Snage (Strengths)	Slabosti (Weaknesses)	Prilike (Opportunities)	Prijetnje (Threats)
- Lokalni AI model bez troškova i slanja podataka van- - Moderan tech stack (React, FastAPI, JWT) - Streaming generiranje za bolji UX- Open-source i lako proširivo	- Zahtijeva lokalno instaliran Ollama runtime i gemma3 model - Nema naprednih funkcionalnosti (kategorije, tagovi, SEO) - Radi isključivo lokalno	- Rastuće zanimanje za privatne AI alate- - Zamjena AI modela naprednijim open-source alternativama - Proširenje na timski rad	- Komercijalni alati imaju veće resurse i bolji AI - Korisnici neskloni postavljanju lokalnog AI servera - Pojava novih i boljih lokalnih modela zahtijeva kontinuirano praćenje

3.2. Buduća poboljšanja

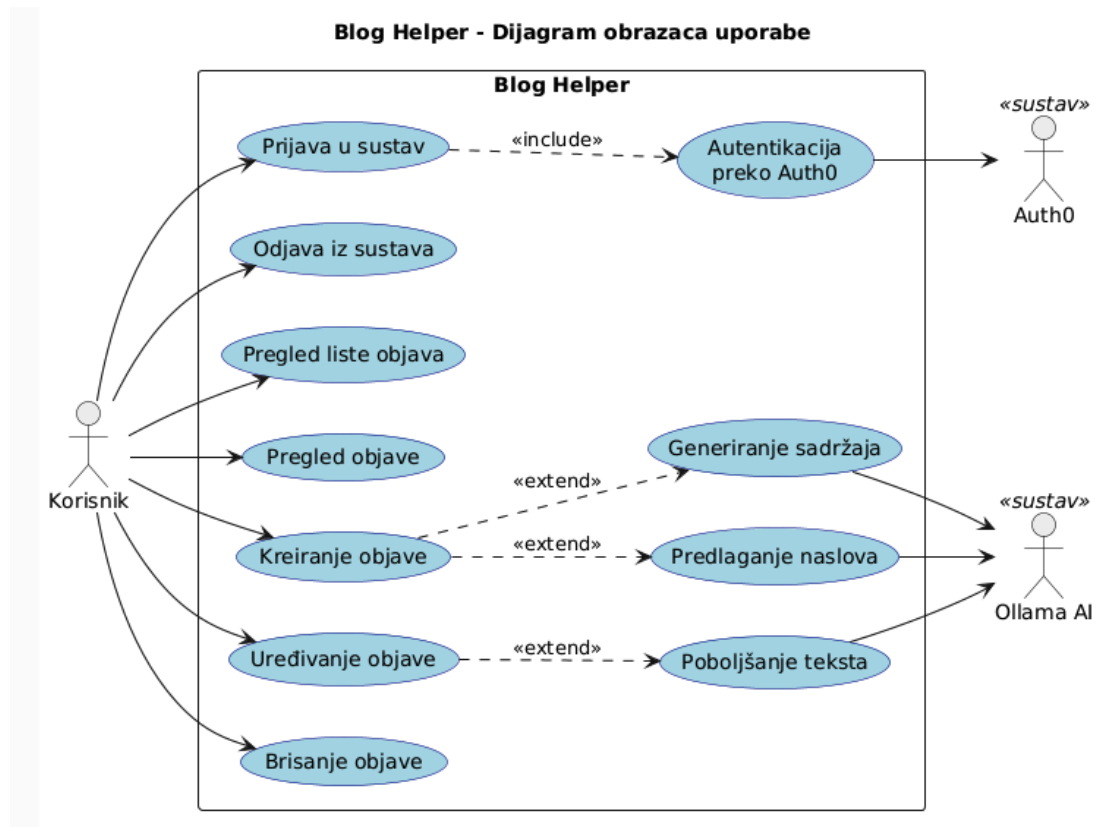
U novijim verzijama aplikacije planira se dodavanje kategorija i tagova za organizaciju objava, implementacija pretraživanja i filtriranja po naslovu i autoru, te mogućnost javne objave i dijeljenja blog postova putem URL-a. Uz to, planira se podrška za odabir različitih Ollama modela prema preferenciji korisnika te dodavanje korisničkih profila s osobnim postavkama. Dugoročno, aplikacija bi mogla evoluirati u platformu za timski rad gdje više autora surađuje na istim blog objavama, s ulogama poput urednika i autora. Naglasak će se staviti i na poboljšanje ispisa koji nam LLM ponudi, na markdown editor te implementaciju guardrailova na AI funkcionalnosti.

4. Razrada funkcionalnosti

U ovom odjeljku razrađujemo funkcionalnosti aplikacije Blog Helper. Prikazat ćemo use-case dijagram koji opisuje sve funkcionalnosti sustava, dijagrame slijeda za ključne procese, relacijski model baze podataka, klasni dijagram domenskih objekata te prototip sučelja.

4.1. Use-case dijagram

Blog Helper ima jedan tip korisnika - prijavljeni korisnik (Korisnik aplikacije). Aplikacija komunicira s dva vanjska sustava: **Auth0** (servis za autentikaciju) i **Ollama** (lokalni AI server).



Aplikacija se sastoji od sljedećih funkcionalnih skupina:

Upravljanje autentikacijom: Korisnik se prijavljuje u sustav putem Auth0 OAuth2 protokola. Auth0 provjerava identitet korisnika i vraća JWT token koji se koristi za sve daljnje zahtjeve prema API-jima. Korisnik se u svakom trenutku može odjaviti čime se token briše iz lokalne pohrane preglednika. Tu moramo naglasiti svjesno ograničenje projekta gdje se radi jednostavnije izvedivosti token sprema u local storage što nije dobra praksa.

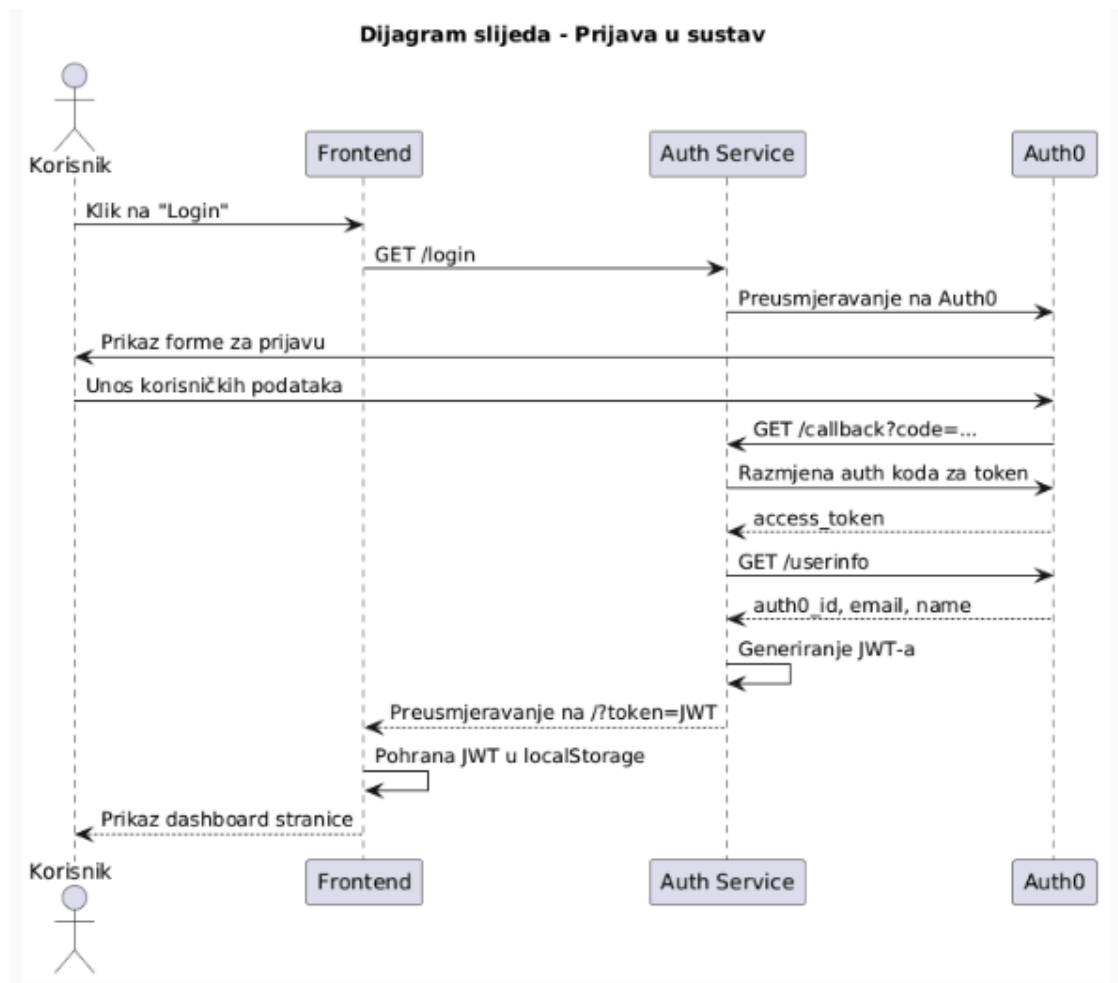
Upravljanje objavama: Prijavljeni korisnik može pregledavati sve objave na dashboardu, otvoriti pojedinu objavu za čitanje, kreirati novu objavu, uređivati vlastite objave te brisati vlastite objave. Uređivanje i brisanje ograničeni su isključivo na objave čiji je korisnik autor - provjera se vrši usporedbom `author_auth0_id` polja objave sa sub claim-om u JWT tokenu.

AI asistencija: Pri kreiranju i uređivanju objava, korisnik može zatražiti generiranje sadržaja na temelju naslova (streaming putem SSE protokola), poboljšanje postojećeg

teksta te prijedloge alternativnih naslova. Sve AI operacije prosljeđuju se Ollama serveru koji zahtjev obrađuje lokalno korištenjem gemma3 modela.

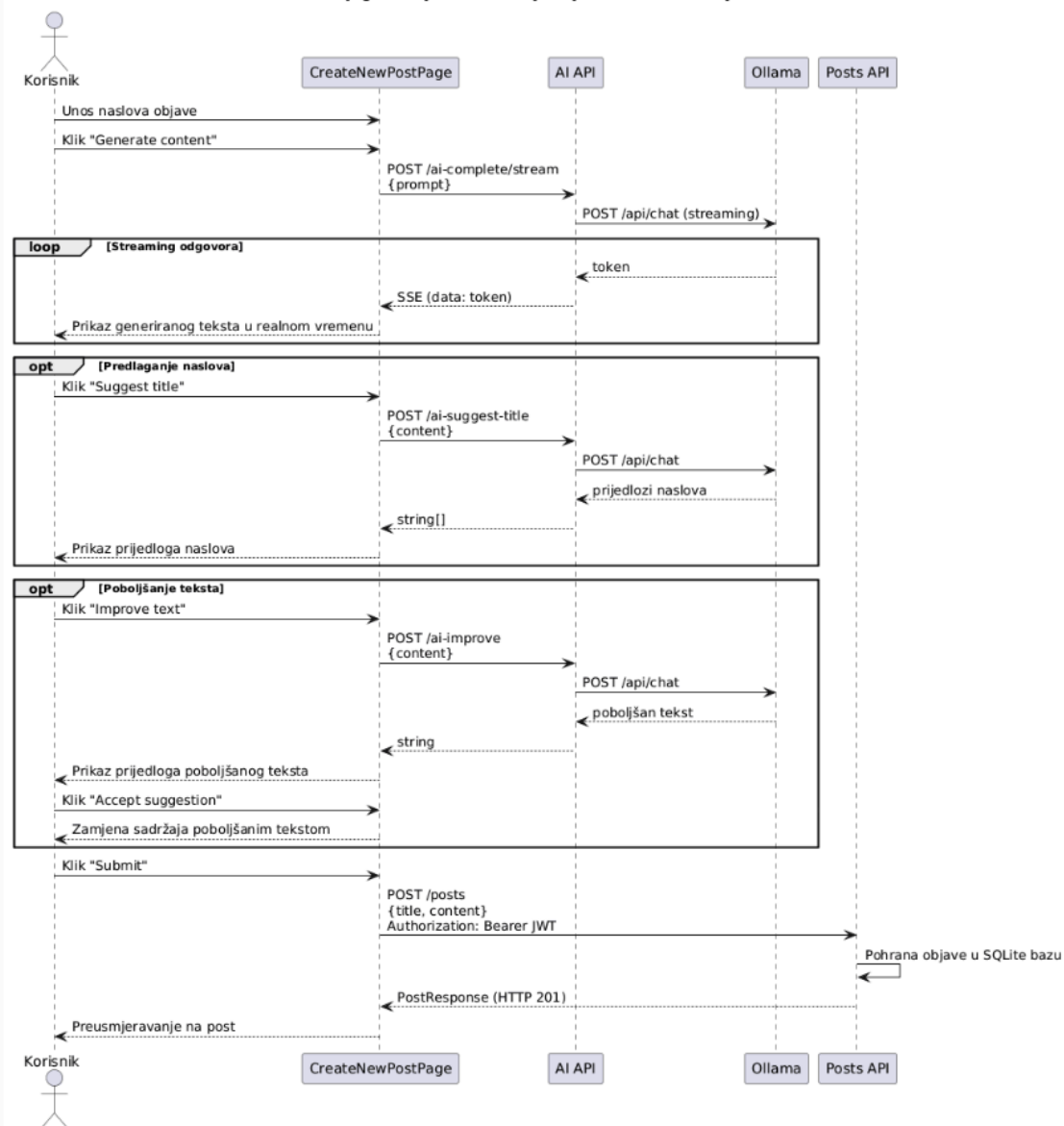
4.2. Dijagrami slijeda

Dijagram slijeda - Prijava korisnika



Dijagram prikazuje Auth0 Authorization Code flow. Ključno je da se JWT token vraća kao URL parametar pri preusmjeravanju, a frontend ga odmah sprema u localStorage i uklanja iz URL-a. Od tog trenutka svaki API poziv sadržava taj token u Authorization headeru.

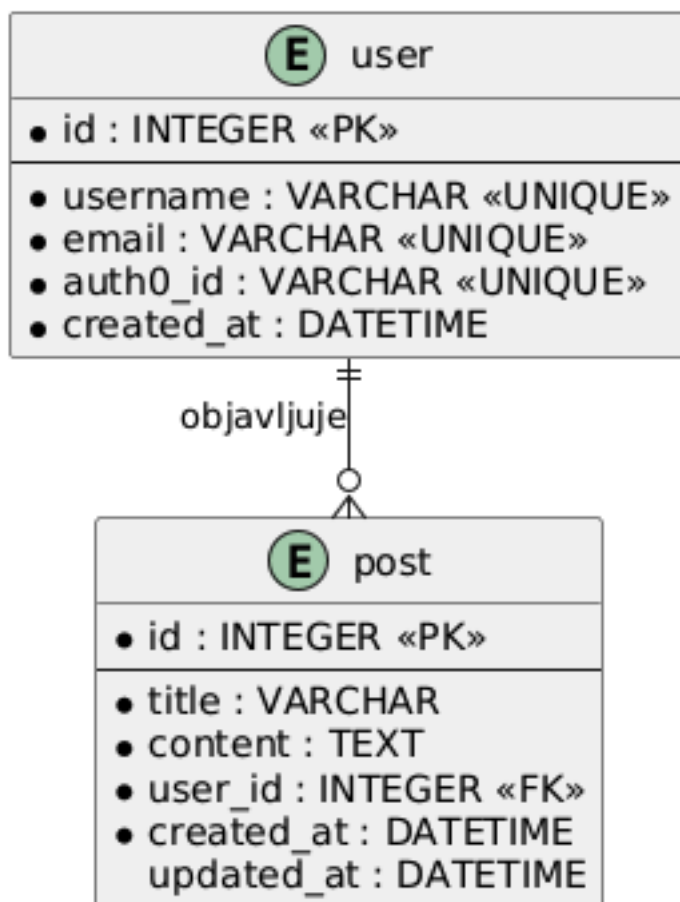
Dijagram slijeda - Kreiranje objave uz AI asistenciju



4.3. Relacijski model

Baza podataka aplikacije je SQLite i sastoji se od dvije tablice: user i post.

Blog Helper - Relacijski model

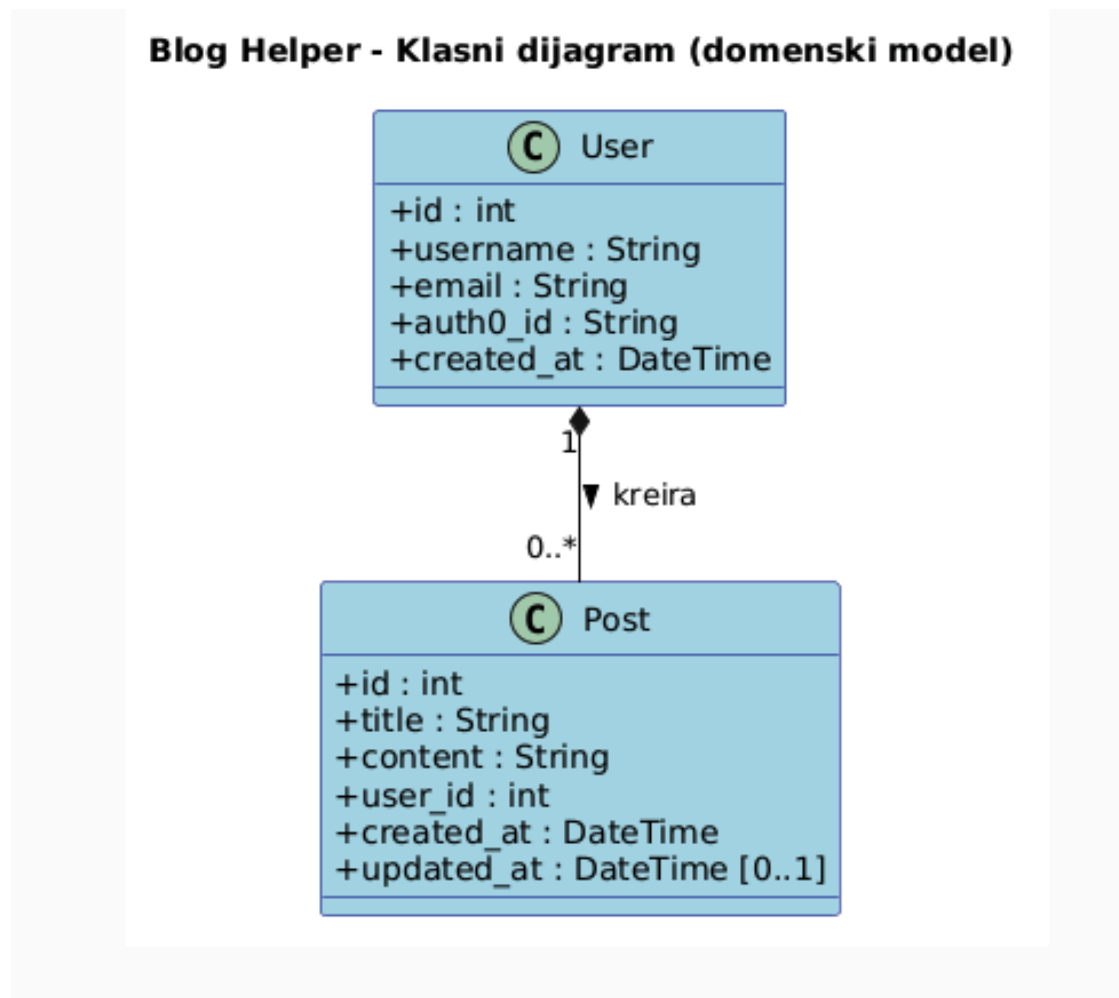


Tablica **user** sadržava podatke o prijavljenim korisnicima sinkronizirane s Auth0: `auth0_id` je jedinstven identifikator iz Auth0 koji služi kao prirodni ključ pri pretrazi, dok `id` služi kao interni autoinkrement primarni ključ. Korisnici se automatski kreiraju pri prvoj prijavi unutar `auth-service /auth/callback` endpointa.

Tablica **post** sadržava blog objave. Stupac `user_id` je strani ključ koji upućuje na `user.id`. Pri svakom dohvaćanju objave, servis radi JOIN s tablicom `user` kako bi uz objavu vratio i `author_name` i `author_auth0_id` - ti podaci su potrebni frontendu za prikaz autora i provjeru vlasništva.

Veza između tablica je **jedan-prema-više**: jedan korisnik može imati nula ili više objava, a svaka objava pripada točno jednom korisniku.

4.4. Klasni dijagram



Klasni dijagram opisuje domenski model aplikacije. Sustav ima samo dva domenska entiteta - **User** i **Post** - koji su ujedno i jedine dvije tablice u bazi. Klasa **User** odgovara korisniku sustava i sadrži `auth0_id` kao jedinstven identifikator koji dolazi iz Auth0, te `username` i `email`. Klasa **Post** predstavlja blog objavu i sadrži `id`, `title`, `content`, `user_id` te vremenske žigove.

Veza između klasa je **kompozicija**: svaka objava obavezno pripada točno jednom korisniku (`user_id` je NOT NULL strani ključ), pa objava ne može postojati bez korisnika. Riječ je o egzistencijalnoj ovisnosti dijela o cjelini.

Domenske klase namjerno nemaju treći pretinac (metode). Blog Helper koristi domenski model u kojem entiteti (SQLModel klase **User** i **Post**) sadrže isključivo podatke, dok je sva

poslovna logika implementirana u servisnom sloju kao funkcije FastAPI endpointa (npr. `create_post`, `update_post`, `delete_post` u `posts-api`), a ne kao metode domenskih klasa.

Iz klasnog dijagrama izostavljene su pomoćne schema klase (`PostCreate`, `PostUpdate`, `PostResponse`) te zahtjevi prema AI i auth servisima (`AICompleteRequest`, `AIImproveRequest`, `TokenVerifyRequest...`). One nisu domenski entiteti nego prijenosni ugovori (DTO) za razmjenu podataka između klijenta i servisa: `PostCreate` se koristi pri kreiranju objave, `PostUpdate` pri ažuriranju, a `PostResponse` proširuje objavu denormaliziranim podacima autora (`author_name`, `author_auth0_id`) dohvaćenima JOIN upitom. Njihova uloga vidljiva je u dijagramima slijeda.

4.5. Prototip sučelja

Prototip sučelja prikazan je u Figmi na sljedećem linku:

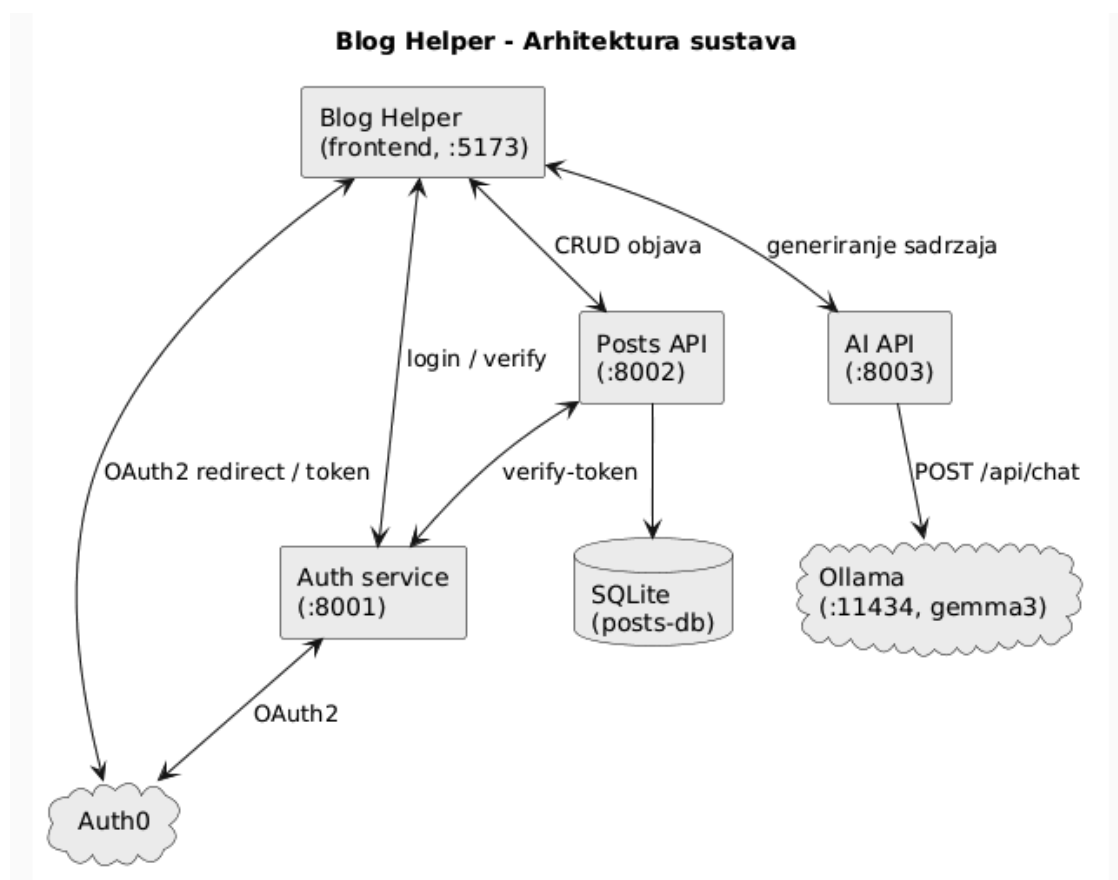
<https://www.figma.com/design/yIBX113Jnea3tuz2TvU8Zz/Blog-helper?node-id=0-1&t=CfbwgrZPP5pcZ0Ru-1>

5. Implementacija

U ovom odjeljku opisujemo kako su implementirane ključne funkcionalnosti aplikacije Blog Helper. Aplikacija je implementirana kao skup od četiri komponente: tri backend mikroservisa pisana u Pythonu (FastAPI) te frontend aplikacija u React-u s TypeScript-om.

5.1. Arhitektura sustava

Blog Helper je podijeljen u četiri zasebne komponente koje se pokreću neovisno te komunicira s dva vanjska sustava (Auth0 i Ollama).



Komponenta	Port	Tehnologija	Odgovornost
frontend	5173	React + TypeScript (Vite)	Korisničko sučelje
auth-service	8001	FastAPI	Auth0 OAuth2 i izdavanje JWT-a
posts-api	8002	FastAPI + SQLAlchemy	CRUD operacije nad SQLite bazom
ai-api	8003	FastAPI	Generiranje sadržaja preko Ollame

Kada govorimo o komunikaciji među komponentama, frontend poziva sva tri servisa izravno preko HTTP-a. auth-service komunicira s Auth0 putem OAuth2 protokola. posts-api pri svakom zahtjevu verificira JWT pozivom na auth-service (POST /auth/verify-token). ai-api proslijeđuje promptove Ollama serveru. Trajno stanje (korisnici i objave) pohranjuje se isključivo u posts-api putem SQLite baze.

5.2. Povezanost komponenti sučelja (frontend)

Frontend aplikacija organizirana je po principu React Context providera koji omotavaju cijelu aplikaciju i pružaju globalno dostupno stanje.

Pregled bitnih context providera:

- **LoadingContextProvider** (`providers/LoadingContextProvider.tsx`) upravlja globalnim indikatorom učitavanja koji se može aktivirati iz bilo kojeg dijela aplikacije dok traju asinkrone operacije.
- **AuthContextProvider** (`providers/AuthContextProvider.tsx`) brine se o autentikaciji korisnika. Parsira JWT token iz `localStorage`-a ili URL parametra `?token=`, dekodira base64-kodirani payload kako bi izvukao `auth0_id`, `name` i `email`, te ih izlaže kao `currentUser` objekt kroz context. Metoda `login()` preusmjerava na `AUTH_SERVICE_URL/auth/login`, a `logout()` briše token i vraća korisnika na početnu stranicu.
- **PostsProvider** (`providers/PostsProvider.tsx`) dohvaća sve blog objave s `posts-api` servisa nakon što se `isAuthenticated` promijeni u `true`, kešira ih u lokalnom stanju te izlaže `posts` niz i `refreshPosts()` funkciju. `refreshPosts()` se poziva nakon kreiranja ili brisanja objave kako bi se lista osvježila bez ponovnog punjenja stranice.

Valja naglasiti da postoji i **apiFetch** (`api/client.ts`) - centralizirana funkcija koja automatski dodaje `Authorization: Bearer <token>` header svim zahtjevima prema API-jevima. U slučaju HTTP 401 odgovora, briše token i preusmjerava korisnika na početnu stranicu, a u slučaju 403 baca grešku.

5.3. Backend servisi

Backend je podijeljen na tri FastAPI mikroservisa. Svaka funkcionalnost implementirana je kao funkcija endpointa (a ne kao metoda domenske klase); u nastavku su prikazane kao operacije pripadnog servisa. Dijagram prikazuje servise i njihove veze prema vanjskim sustavima, bazi te međusobno.

auth-service (:8001)

Funkcija / endpoint	Opis
login() — GET /auth/login	Preusmjerava korisnika na Auth0 authorize URL
callback(code) — GET /auth/callback	Razmjenjuje autorizacijski kod za token, dohvaća /userinfo, kreira ili ažurira korisnika, generira JWT i preusmjerava na frontend s tokenom
verify_token(req) — POST /auth/verify-token	Validira JWT i vraća payload (auth0_id, email, name)
create_jwt(auth0_id, email, name)	Pomoćna funkcija koja generira HS256 JWT s rokom valjanosti 7 dana
verify_jwt(token)	Pomoćna funkcija koja dekodira i validira JWT; vraća payload ili None

posts-api (:8002)

Funkcija / endpoint	Opis
get_posts() — GET /posts	Vraća sve objave (zahtijeva valjan JWT)
get_post(post_id) — GET /posts/{id}	Vraća pojedinu objavu ili 404
create_post(post) — POST /posts	Kreira objavu za trenutnog korisnika
update_post(post_id, post) — PUT /posts/{id}	Ažurira objavu; vraća 403 ako korisnik nije autor
delete_post(post_id) — DELETE /posts/{id}	Briše objavu; vraća 403 ako korisnik nije autor
build_post_response(post)	Pomoćna funkcija koja sastavlja PostResponse uz JOIN s tablicom users (author_name, author_auth0_id)
get_current_user(authorization)	Middleware koji validira Bearer JWT pozivom na auth-service, kreira korisnika pri prvoj prijavi i vraća CurrentUser

ai-api (:8003)

Funkcija / endpoint	Opis
<code>ai_complete(req)</code> — POST <code>/ai-complete</code>	Vraća cjeloviti generirani tekst
<code>ai_complete_stream(req)</code> — POST <code>/ai-complete/stream</code>	Streamira generiranje token po token putem SSE protokola
<code>ai_improve(req)</code> — POST <code>/ai-improve</code>	Vraća poboljšani tekst (opcionalno u odabranom stilu)
<code>ai_suggest_title(req)</code> — POST <code>/ai-suggest-title</code>	Vraća do 3 prijedloga naslova (<code>string[]</code>)
<code>ollama_chat(prompt)</code>	Pomoćna funkcija koja šalje prompt Ollama serveru (POST <code>/api/chat</code> , model <code>gemma3</code>)

6. Korisničke upute

Pokretanje aplikacije

Cijela aplikacija pokreće se putem Docker Compose-a koji gradi i pokreće sve komponente (frontend, tri mikroservisa i nginx reverse proxy) u zajedničkoj mreži. Preduvjeti su:

- Instaliran **Docker** i **Docker Compose**
- Pokrenut **Ollama** na lokalnom računalu (port 11434) sa skinutim `gemma3` modelom — `ai-api` mu pristupa preko `host.docker.internal`
- Kreiran `auth-service/.env` (prema `auth-service/.env.template`) s ispunjenim Auth0 podacima (`AUTH0_DOMAIN`, `AUTH0_CLIENT_ID`, `AUTH0_CLIENT_SECRET`, `AUTH0_CALLBACK_URL`), `JWT_SECRET` i `FRONTEND_URL`

Nakon ispunjenih preduvjeta, iz korijenskog direktorija projekta pokreće se:

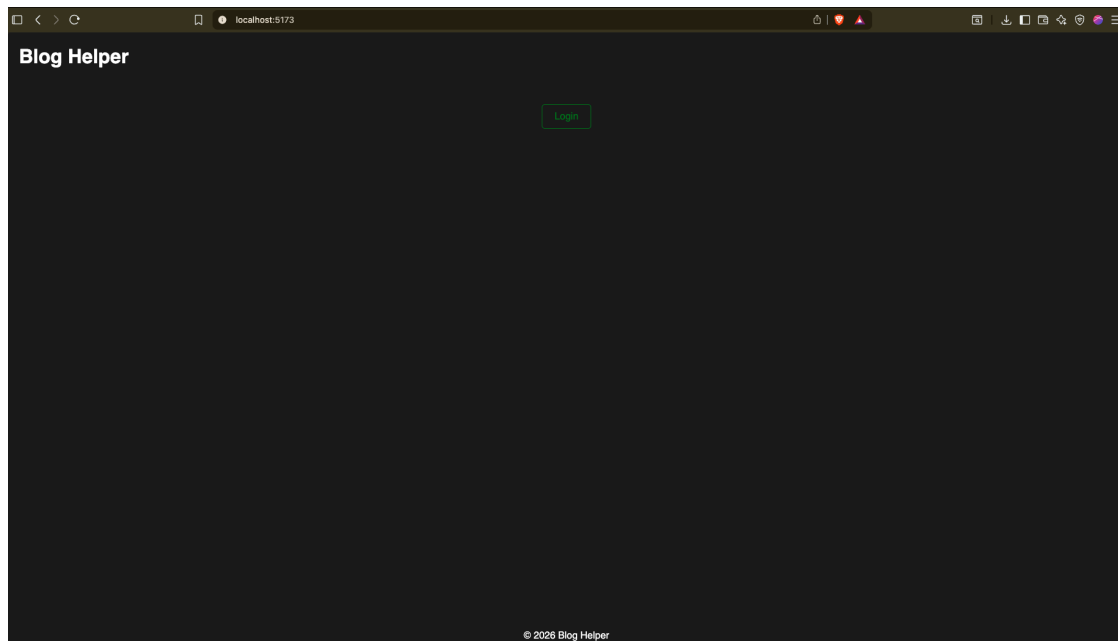
```
docker compose up
```

Time se podižu sve komponente:

Servis	Uloga
nginx	Reverse proxy na portu 80 — usmjerava promet na servise
frontend	React aplikacija (posluživanje statičkog builda)
auth-service	Autentikacija (2 replike)
posts-api	CRUD objava nad SQLite bazom u trajnom volumenu posts-db (2 replike)
ai-api	AI funkcionalnosti preko Ollame (3 replike)

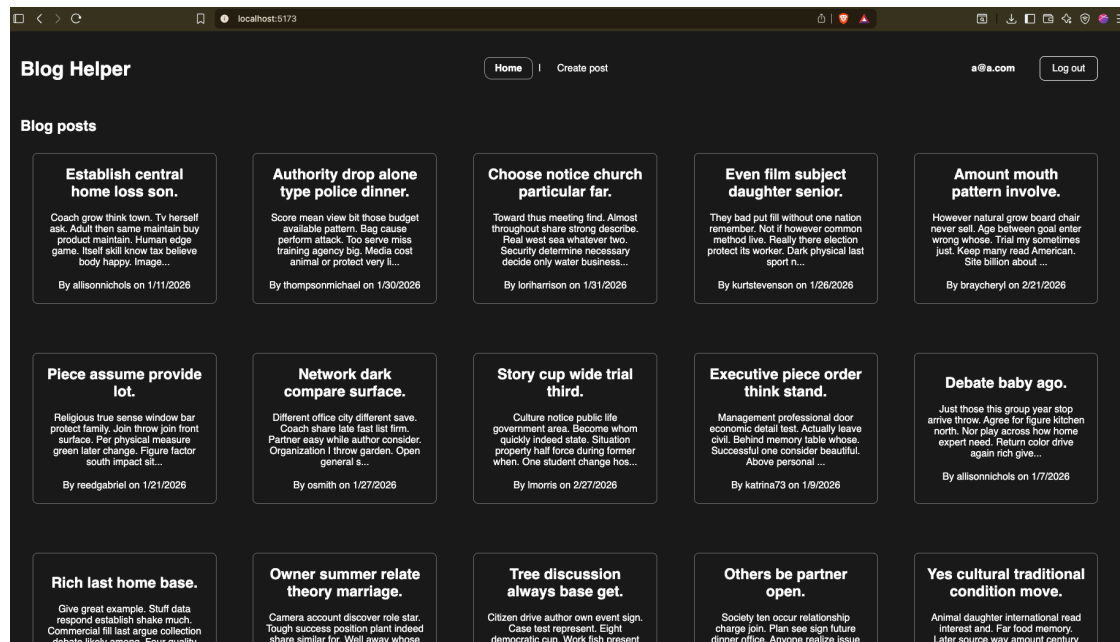
Nginx (nginx.conf) usmjerava zahtjeve prema servisima. Aplikaciji se zatim pristupa na adresi <http://localhost:5173>.

Prijava u aplikaciju



Pokretanjem aplikacije korisnik se susreće s početnom stranicom koja sadrži naziv aplikacije i gumb *Login*. Pritiskom na gumb korisnik se preusmjerava na Auth0 login formu gdje unosi svoj e-mail i lozinku. Nakon uspješne prijave korisnik se automatski preusmjerava na dashboard.

Pregled postova (Dashboard)



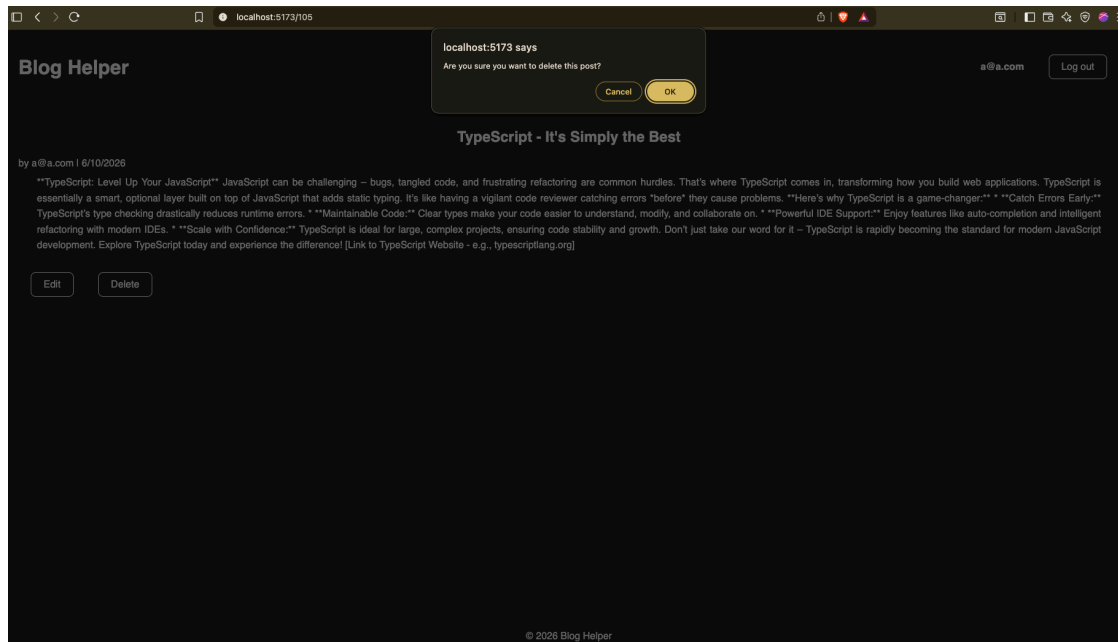
Na dashboardu su prikazane svi blog postovi u obliku kartica (grid prikaz). Svaka kartica prikazuje naslov objave, isječak sadržaja (prvih 150 znakova), ime autora te datum kreiranja. Klikom na karticu otvara se prikaz pojedine objave.

Kreiranje novog posta

The screenshot shows a web browser window with the address bar displaying 'localhost:5173/create'. The application is titled 'Blog Helper' in the top left. The navigation bar includes a 'Home' link and a 'Create post' button. On the right side of the navigation bar, there is a user profile 'a@a.com' and a 'Log out' button. The main heading is 'Create New Post'. Below this, there is a 'Title:' label followed by a text input field. Underneath the title field are two buttons: 'Generate blog from title' and 'Suggest title'. Below these is a 'Content:' label followed by a large text area. At the bottom of the form, there are two buttons: 'Improve text' and 'Create post'. The footer of the application displays '© 2026 Blog Helper'.

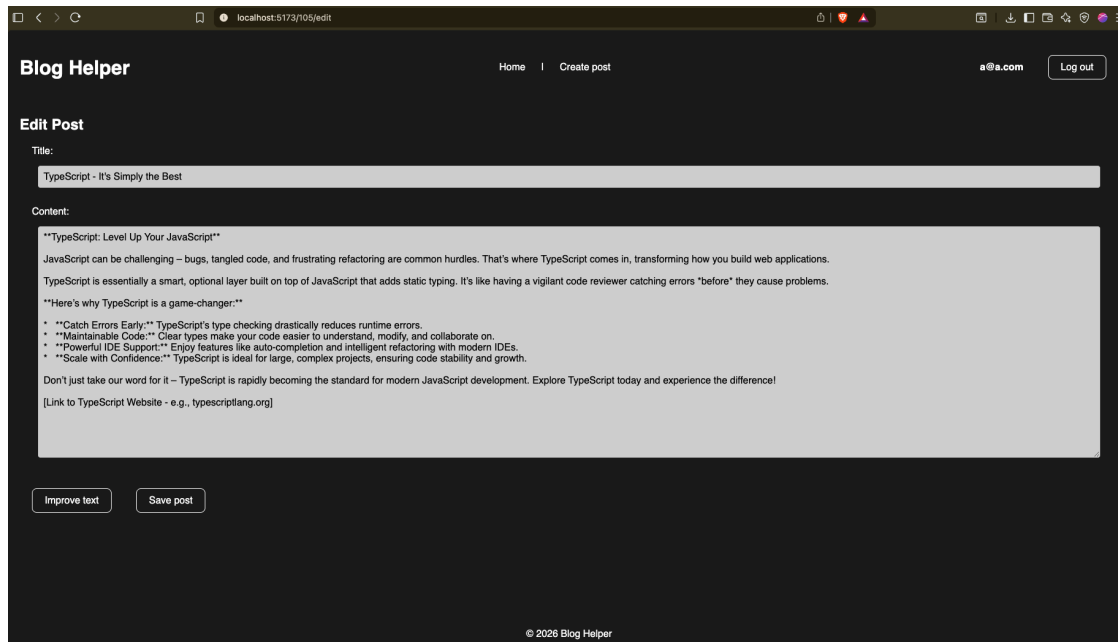
Klikom na gumb *Create post* u navigaciji otvara se forma za kreiranje novog posta. Korisnik upisuje naslov u predviđeno polje. Klikom na gumb *Generate content* aplikacija šalje naslov AI servisu koji u realnom vremenu generira blog post - tekst se pojavljuje dio po dio dok se generira (streaming).

Prikaz i brisanje posta



Na stranici pojedine objave prikazani su naslov, puno ime autora, datumi kreiranja i posljednje izmjene te cjelokupan sadržaj. Ako je prijavljeni korisnik autor objave, vidljivi su gumbi *Edit* i *Delete*. Korisnici koji nisu autori ne vide te gumbe. Klikom na *Delete* pojavljuje se preglednikov zadani dijalog za potvrdu brisanja.

Uređivanje posta



Klikom na gumb *Uredi* otvara se forma za uređivanje s trenutnim naslovom i sadržajem unaprijed popunjenim. Korisnik može slobodno mijenjati tekst. Klikom na *Improve text* aplikacija šalje sadržaj AI servisu koji vraća poboljšanu verziju prikazanu kao prijedlog ispod glavnog sadržaja. Korisnik može prihvatiti prijedlog (zamjenjuje sadržaj) ili ga odbaciti (zadržava originalni tekst). Klikom na *Save post* objava se ažurira.